1 / MCU: od zadání k programu

Maturitní přehled - programování jednočipových mikropočítačů a PLC. Zkrácení osmistránkového podkladu; důraz na výklad principu, čtení kódu a praktické použití.

Jak vzniká firmware

MCU je mikrokontrolér. Program čte vstupy, rozhoduje, ovládá výstupy a komunikuje. Postup: zadání a seznam I/O → algoritmus nebo stavový automat → zdrojový kód → překlad a sestavení → nahrání → ladění a test.

Kompilátor překládá kód, linker spojuje části programu a knihovny. Cross-kompilace probíhá na PC pro jiný procesor. Firmware je program zařízení; bootloader umožňuje jeho nahrání. Knihovna dodává hotové funkce, API jejich rozhraní, framework strukturu a služby.

Jazyky a vývojová prostředí

Jazyk / prostředí	Co si zapamatovat
ASM, C, C++	ASM: instrukce a registry; C: efektivní řízení MCU; C++: navíc třídy a šablony. Arduino používá C++.
MicroPython, bloky	Interpretované prostředí pro podporované desky; bloky např. pro výuku.
Arduino IDE	Sketch .ino, desky a knihovny, Verify, Upload, Serial Monitor/Plotter.
PlatformIO + VS Code	Projekty a konfigurace platformio.ini, knihovny, build a upload.
ESP-IDF / další IDE	ESP-IDF: framework pro ESP32. Microchip Studio/MPLAB X: podporované AVR/PIC/SAM; Thonny: MicroPython.

Verify/Build = přeložit a sestavit. Upload/Flash = nahrát. Debug = hledat chyby pomocí logů, breakpointů, krokování a sledování proměnných. Sériový monitor nenahrazuje hardwarový debugger; nahrávání a ladění může používat ISP/JTAG/SWD podle čipu.

Minimum C/C++

Typy: bool, char, int, float; uint8_t/uint16_t/uint32_t mají pevnou šířku. const označuje konstantu, lokální static zachovává hodnotu mezi voláními. Rozliš lokální a globální proměnné. Funkce má parametry a případně návratovou hodnotu.

Operátory: = přiřazení, == rovnost, != nerovnost, && AND, || OR, ! NOT, % zbytek. Větvení: if/else, switch/case. Cykly: for, while, do…while (proběhne alespoň jednou).

Události a stavový automat

Polling = pravidelná kontrola vstupu. Přerušení vyvolá ISR: ta má být krátká, typicky jen nastaví příznak. volatile samo nezajistí atomický přístup ani synchronizaci. Debounce potlačuje zákmity tlačítka, watchdog může resetovat zaseknutý systém.

Stavový automat určuje stavy, výstupy a podmínky přechodů: ČEKÁ → po Start BĚŽÍ → po Stop ČEKÁ; při chybě PORUCHA. V C++ lze použít enum a switch, v PLC kroky a přechody.

2 / Arduino a ESP32: klíčové příklady

setup() se provede jednou po startu, loop() se opakuje. delay() blokuje pokračování této smyčky; millis() umožňuje průběžně obsluhovat další vstupy.

Neblokující blikání - Arduino Nano s ATmega328P

```
const uint8_t LED = 13;
uint32_t predchozi = 0;
bool stav = false;
void setup() { pinMode(LED, OUTPUT); }
void loop() {
  uint32_t ted = millis();
  if ((uint32_t)(ted - predchozi) >= 500) {
    predchozi = ted;
    stav = !stav;
    digitalWrite(LED, stav);
  }
}
```

Změna stavu po 500 ms znamená celou periodu přibližně 1 s (1 Hz). Rozdíl neznaménkových časů funguje i přes přetečení millis(), pokud interval nepřekročí rozsah čítače a podmínku kontrolujeme dostatečně často.

GPIO, ADC, PWM a komunikace

pinMode nastavuje INPUT, OUTPUT nebo INPUT_PULLUP. digitalRead/digitalWrite čtou a nastavují logický stav. U INPUT_PULLUP je tlačítko proti GND aktivní při LOW. analogRead čte ADC; analogWrite na klasickém Nano vytváří PWM, nikoli spojité napětí. Serial.begin/print/available slouží ke komunikaci; attachInterrupt registruje ISR. millis/micros měří čas v ms/µs.

```
// Nano AVR: A0 = senzor, D9 = PWM vystup
int hodnota = analogRead(A0);
analogWrite(9, hodnota > 600 ? 200 : 0);
```

Nano AVR: ADC obvykle 0–1023 (10 bitů), PWM 0–255. Střída přibližně D = hodnota/255; pro 128 asi 50 %. Kolem prahu přidej hysterezi. Hardwarový časovač počítá impulzy a vytváří periodické události. UART je asynchronní, I²C adresovaná dvouvodičová sběrnice, SPI synchronní přenos s hodinami a výběrem zařízení.

ESP32 proti klasickému Nano

Nano ATmega328P typicky používá 5V logiku; ESP32 typicky 3,3V a GPIO nejsou obecně 5V tolerantní. ESP32 je rodina čipů: Wi-Fi, Bluetooth, piny, ADC i další periferie závisí na variantě. PWM lze tvořit periferií LEDC; konkrétní API závisí na verzi Arduino core.

ESP32 lze programovat v Arduino IDE/PlatformIO nebo ESP-IDF; některé varianty podporují MicroPython. ESP-IDF používá FreeRTOS: úlohy, priority, fronty, semafory a časovače. Sdílená data vyžadují synchronizaci; také Arduino core ESP32 využívá systémové služby.

```
#include <WiFi.h>
void setup() {
  Serial.begin(115200);
  WiFi.begin("MOJE_WIFI", "HESLO"); // pouze zastupne udaje
}
void loop() {
  // Stav spojeni kontroluj pomoci WiFi.status().
  // Rizeni nesmi cekat v nekonecne smycce na Wi-Fi.
}
```

V praxi doplň timeout a opětovné připojení. Hesla nepatří do veřejného repozitáře.

3 / PLC, Mosaic a jazyky IEC

Jak PLC vykonává program

PLC typicky opakuje: načtení vstupů do obrazu I/O → vykonání programu → aktualizace výstupů → další cyklus. Detaily a další úlohy určuje runtime a konfigurace. Důležité jsou předvídatelná doba reakce, diagnostika a bezpečný stav.

MCU: setup/loop, registry, knihovny, přerušení a případně RTOS. PLC: cyklické i jiné úlohy, proměnné navázané na průmyslové I/O a funkční bloky. Obě řešení potřebují algoritmus, časování, testy a dokumentaci.

Mosaic (v podkladu „Mosaik“) - postup

Vývojové prostředí Teco pro PLC Tecomat. Založ projekt a vyber PLC → nastav moduly a komunikaci → deklaruj proměnné a přiřaď fyzické I/O → napiš program → přelož → připoj se a nahraj → v online režimu sleduj proměnné a diagnostiku.

Typy: BOOL, INT, REAL, TIME. Urči lokální/globální rozsah, počáteční hodnoty a retenci (uchování po restartu). Rozliš program, funkci a funkční blok: instance funkčního bloku uchovává vlastní stav, např. průběh časovače.

IEC 61131-3: různé zápisy stejné logiky

Zápis	Princip a použití
LD - Ladder Diagram	Kontaktní schéma podobné reléové logice; podmínky a blokování.
FBD - Function Block Diagram	Propojené funkční bloky; logika, regulace, časovače.
ST - Structured Text	Text podobný Pascalu; podmínky, výpočty, algoritmy.
SFC - Sequential Function Chart	Kroky, přechody a akce pro strukturování sekvencí.
IL - Instruction List	Starší instrukční zápis; může zůstávat v nástrojích výrobce. Současný přehled IEC 61131-3:2025 uvádí ST, LD, FBD a prvky SFC.

```
LD:  |----[ A ]----[ B ]--------( Q )----|
FBD: A, B -> blok AND -> Q
ST:  Q := A AND B;
```

Výstup je TRUE pouze pro A = TRUE a B = TRUE. LD popisuje programovou logiku; není to schéma fyzického zapojení.

Příklad ST: samodrž s předností Stop

```
VAR
  Start, StopPozadavek : BOOL;
  Motor : BOOL := FALSE;
END_VAR
IF StopPozadavek THEN
  Motor := FALSE;
ELSIF Start THEN
  Motor := TRUE;
END_IF;
```

Bez Start i Stop se Motor nemění. Při obou signálech současně vyhraje Stop. Fyzické vstupy/výstupy musíš namapovat; restart a retenci nastavit záměrně. Běžná programová podmínka Stop nenahrazuje bezpečnostní zastavení.

4 / Časovače, úloha a opakování

Blok	Chování
TON - zpožděné zapnutí	Q sepne po souvislém IN = TRUE po dobu PT; při IN = FALSE se běžně resetuje.
TOF - zpožděné vypnutí	Q sepne s IN; po poklesu IN zůstane sepnutý ještě PT.
TP - impulz	Náběžná hrana spustí impulz délky PT.
CTU - čítač nahoru	Náběžné hrany CU zvyšují CV; Q při dosažení PV; R resetuje.
R_TRIG / F_TRIG	Detekce náběžné / sestupné hrany.

IN = vstup, PT = nastavený čas, Q = výstup, ET = uplynulý čas. Bloky mají vlastní instance a v cyklické úloze se pravidelně volají.

```
VAR
  Casovac : TON;
  Tlacitko, Motor : BOOL;
END_VAR
Casovac(IN := Tlacitko, PT := T#5s);
Motor := Casovac.Q;
```

Tlačítko držené 3 s Motor nezapne. Při nepřerušeném držení 5 s sepne Q; skutečná aktualizace výstupu závisí na cyklu PLC.

Řešená úloha: světlo s doběhem 30 s

Vstupy Pohyb a Tma, výstup Svetlo. Za tmy a při pohybu má svítit; po odeznění podmínky ještě 30 s. Základní varianta z podkladu neřeší poruchu čidla.

```
VAR
  Dobeh : TOF;
  Pohyb, Tma, Podminka, Svetlo : BOOL;
END_VAR
Podminka := Tma AND Pohyb;
Dobeh(IN := Podminka, PT := T#30s);
Svetlo := Dobeh.Q;
```

Pohyb skončí v t = 10 s → vypnutí přibližně v t = 40 s. Nový pohyb v t = 25 s doběh přeruší; skončí-li v t = 28 s, vypnutí bude přibližně v t = 58 s. Pokles Tma také spustí doběh. Pro okamžité vypnutí za světla by byla potřeba jiná logika. U MCU stejnou úlohu řeší millis() a uložený čas poslední události.

Samostatně a kontrolní otázky

Úloha: TON má PT = 5 s. IN je TRUE od 0 do 3 s, znovu od 4 do 10 s. Kdy sepne a vypne Q? Řešení: první pokus se resetuje, Q sepne přibližně v 9 s a vypne při vyhodnocení IN = FALSE v 10 s.

Vysvětli: zdrojový kód × firmware; Verify × Upload × Debug; millis × delay; ADC × PWM; polling × ISR; Arduino core × ESP-IDF; cyklus PLC; LD/FBD/ST/SFC; TON × TOF; samodrž; retence a restart.

Chyby, testování a osnova výkladu

Nezaměň = a ==, PWM a DAC, TON a TOF. Nepředpokládej stejné piny všech ESP32. Testuj mezní hodnoty, odpojené čidlo, ztrátu Wi-Fi, přetečení času, restart a definovaný stav výstupů. Použij logy, debugger, simulaci a měření signálů; verzuj projekt.

Výklad na 15 minut: 0–3 min vznik programu a prostředí; 3–7 min Arduino, časování a ESP32; 7–11 min PLC, Mosaic a IEC zápisy; 11–15 min časovače, příklad, diagnostika a porovnání MCU/PLC.

Zdroj: přiložený podklad „Programování MCU a PLC“, 8 stran. Název Mosaic a přehled IEC ověřeny u Teco a IEC; formulace o IL upravena. Příklady jsou výukové a vyžadují konfiguraci konkrétního zařízení.